

Start

clear clap detector sensor data

forever

wait until clap detected

clear clap detector sensor data

wait 0.2 sec

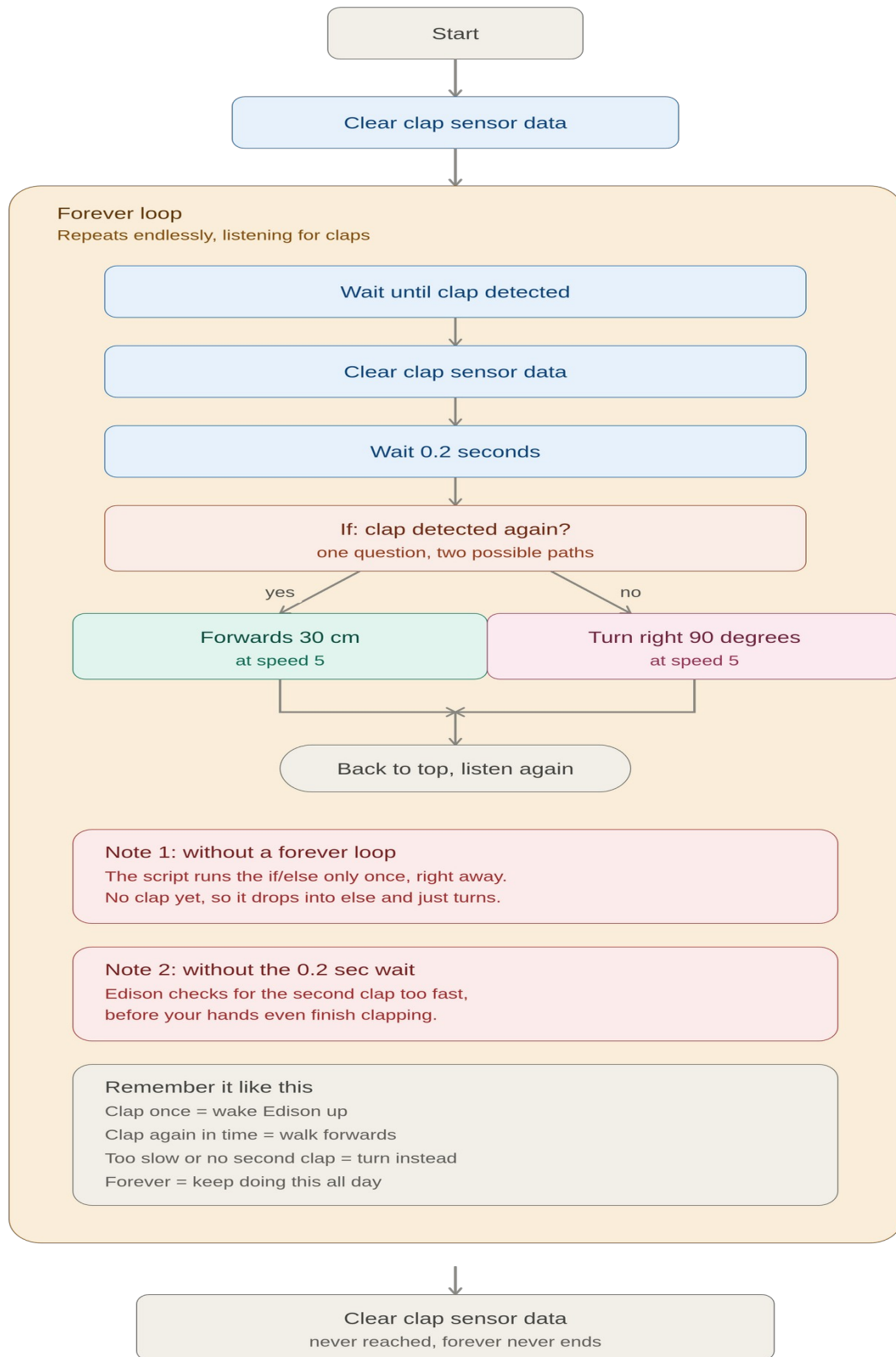
if clap detected then

forwards for 30 cm at speed 5

else

turn forwards right for 90 degrees at speed 5

clear clap detector sensor data



Start — Edison wakes up and gets ready to run the program.

Clear clap detector sensor data — Edison wipes its "ears" clean, like erasing a whiteboard, so old sounds don't confuse it.

Forever — this is a loop that never stops. Everything inside it keeps repeating, again and again, for as long as the program runs.

Wait until clap detected — Edison just stands still and listens. It won't do anything else until it hears a clap.

Clear clap detector sensor data (again) — as soon as it hears that first clap, it wipes its ears clean again, so it's ready to listen for a *second* clap without the first one confusing it.

Wait 0.2 seconds — Edison pauses for a fifth of a second. This gives your hands time to clap a second time.

If clap detected... then... else... — this is the big decision. Edison asks one question: *"Did I hear a second clap in that little pause?"*

- If **yes** → it goes forwards 30 cm at speed 5. One path.
- If **no** → it turns right 90 degrees at speed 5. The other path.

Only **one** of these two things happens each time — never both. That's what makes it an if/else: it's a fork in the road, not a checklist.

Back to the top of forever — after moving or turning, Edison loops back up to "wait until clap detected" and starts listening all over again.

Clear clap detector sensor data (the very last block) — this sits *outside* the forever loop. Since forever never actually finishes, Edison never reaches this block while the program is running.

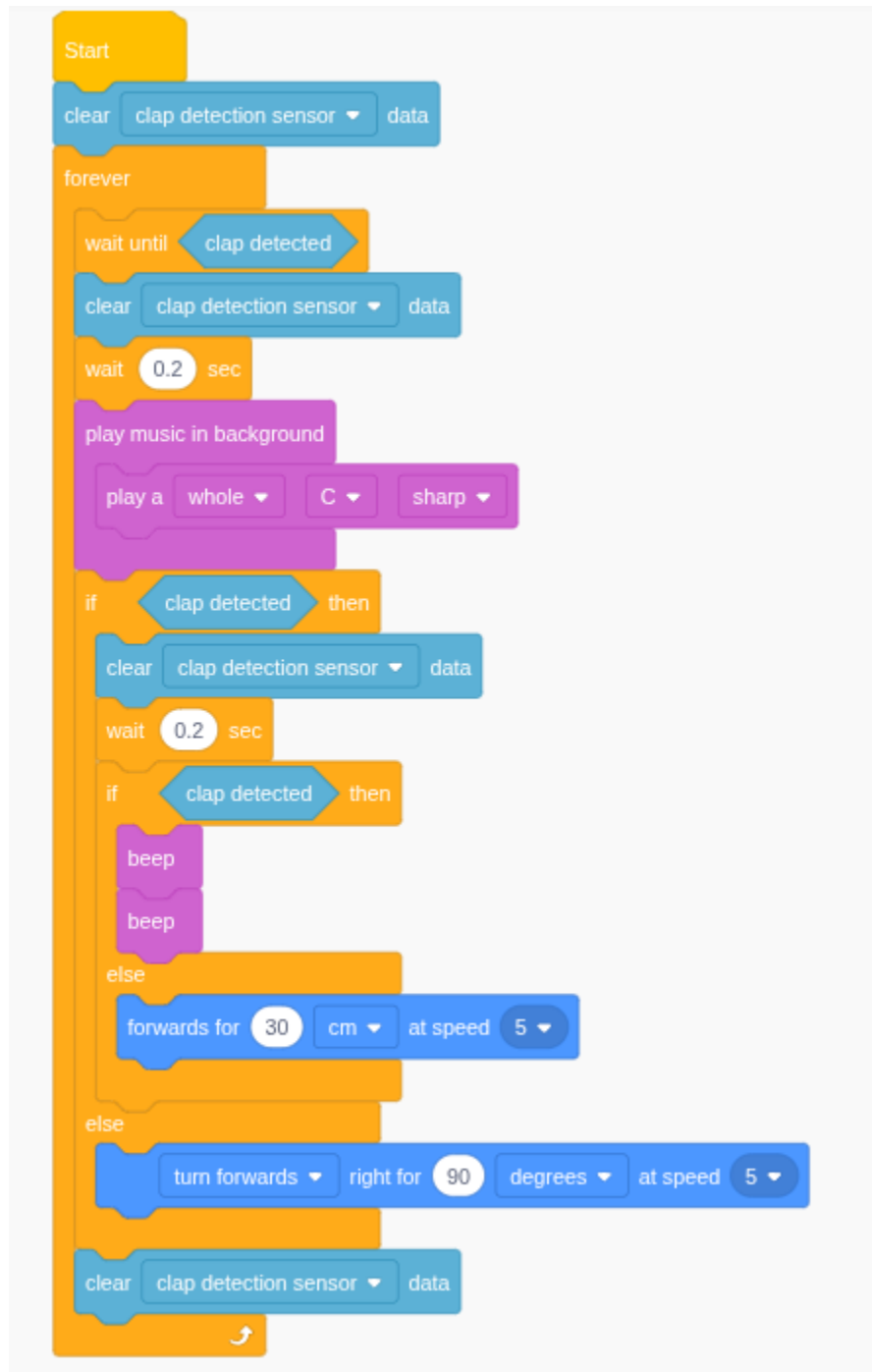
The two important gotchas

No forever loop → no repeating. If you took the forever block away, Edison would run the if/else exactly one time, the instant the program starts. Since no one has clapped yet, "clap detected" is false immediately, so it falls straight into else and just turns 90 degrees — then the whole program is done. It can't wait around for a first clap because there's nothing telling it to keep checking.

No wait → no time to clap twice. Computers check things incredibly fast, way faster than a human can clap twice in a row. Without that little `wait 0.2 sec` pause, Edison would check for the second clap before your hands even finish moving apart from the first clap. The wait is what gives your second clap a chance to be heard.

Extension

The 3-clap version



Same skeleton as before, just one more layer added onto the existing if/else:

- clear sensor → forever
- wait until clap detected (clap 1) → clear sensor → wait 0.2 sec

- **if** clap detected again (clap 2)?
 - **then** → `clear sensor` → `wait 0.2 sec` → **if** clap detected a 3rd time?
 - **then (3 claps)** → beep + flash LED
 - **else (2 claps)** → forwards 30 cm at speed 5
 - **else (1 clap)** → turn right 90° at speed 5
- `clear sensor` → loop back to top

The trick: your old if/else didn't disappear — it just moved inside the "then" branch of a *new* outer if/else. Every extra clap you want to count means nesting one more if/else one level deeper, and giving Edison one more `clear sensor` + `wait 0.2 sec` pair to catch that next clap.

The 4-clap version (harder mode)

Same idea, one level deeper again:

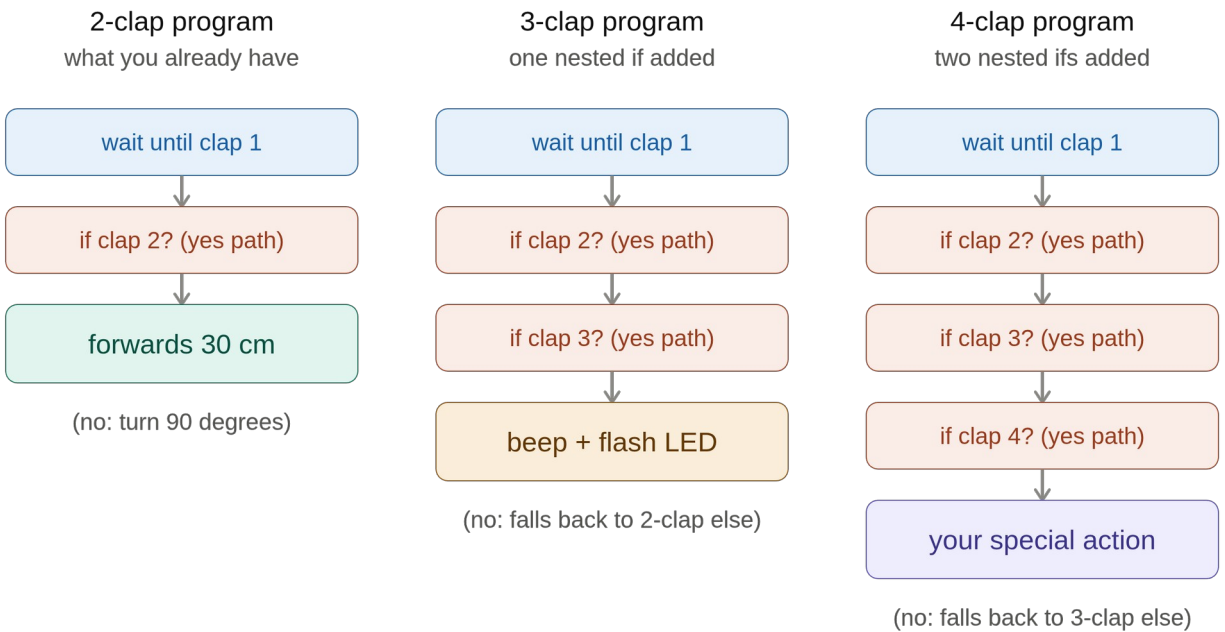
- clap 1 → wait/listen
- **if** clap 2? → **if** clap 3? → **if** clap 4?
 - **then (4 claps)** → your special action
 - **else (3 claps)** → beep + flash
 - **else (2 claps)** → forwards
- **else (1 clap)** → turn

By 4 claps you've got an if buried three levels inside another if inside another if — technically fine, but genuinely annoying to read and edit. This is the point where a **clap counter variable** is the smarter build: make a variable called `claps`, add 1 to it every time `wait until clap detected` fires, and reset it to 0 whenever there's a pause longer than, say, 1 second with no clap. Then one single check — "is `claps` equal to 4?" — replaces the whole nested pyramid.

That picture shows the "happy path" (every clap heard in time) for all three programs side by side. A couple of things to notice comparing the columns:

- Each program is the previous one with **exactly one more** `if clap N?` `then ... else ...` **bolted onto the bottom of the last "then"**. Nothing else changes.
- Before every new `if`, there's a `clear sensor` + `wait 0.2 sec` pair (left off the picture above to keep it readable — it sits right before each "if clap N?" box).
- Every "yes" box also has an "else" hanging off it that drops back down to the shorter program's action — that's the part not drawn here, since it's identical to what the very first diagram showed you, just one level deeper each time.

So the 4-clap program, in full, reads like: *clap 1* → *if clap 2* → *if clap 3* → *if clap 4* → *special action, else beep+flash, else forwards, else turn*. Four nested nesting levels, four different possible outcomes, one program.



Every yes box in this picture also has an else underneath it (shown in the earlier diagram) that falls back to the shorter program.

Start

clear clap detector sensor data

forever

wait until clap detected

clear clap detector sensor data

wait 0.2 sec

if clap detected then

clear clap detector sensor data

wait 0.2 sec

if clap detected then

play beep

play beep

else

forwards for 30 cm at speed 5

else

turn forwards right for 90 degrees at speed 5

clear clap detector sensor data (never reached, forever loops)

TEST

2-clap program:

1. c lap — single clap, then wait. (turn 90 degree)
2. c lap c lap, ~0.2 sec apart (move forward)

3-clap program:

1. c lap — single clap, then wait.
2. c lap c lap, ~0.2 sec apart — this only exercises the *outer* if (clap 1 vs clap 2), same as the 2-clap test.
3. c lap ~0.2 sec c lap ~0.5 sec c lap — first gap stays short , and only the *second* gap widens to ~0.5 sec . So it's uneven spacing, not clap...clap...clap all equally 0.5 apart: **clap-clap** close together, then a slightly longer beat before the third clap.